Модуль: аутентификация и учётные записи

1. Назначение и границы

Пользователи, пароли (хеширование), JWT (подпись/проверка), сессии и кэш сессий. Изолирован от аналитики: живёт в отдельной SQLite-базе. Не делает: не управляет правами на предметные данные (это решает API-слой), не хранит пароли в открытом виде.

2. Сценарии использования

- Создание пользователя из CLI (`python -m backend.users create …`).
- Вход по паролю, выдача токена, ограниченное время жизни (`ACCESS_TOKEN_EXPIRE_MINUTES`).
- Проверка токена на каждом запросе; истечение — 401.
- Отказ: неверный пароль и отсутствующий пользователь отвечают одинаково нейтрально.

3. Контракт

`/auth/*` (login, refresh, logout) — в `backend/openapi.json`. Внутренние контракты: `backend/auth/service.py` (`AuthService`, `Session`, `AuthorizationRequest`).

4. Данные

Владеет SQLite-схемой аутентификации (`backend/auth_schema.sql`): пользователи, сессии, ключи. PostgreSQL не используется — это делает модуль независимо тестируемым.

5. Права

Модуль отличает «аутентифицирован» от «авторизован». Роли и доступ к аналитике проверяются в API-слое. Приватный ключ JWT — файл из `AUTH_PRIVATE_KEY`, права 0600.

6. Зависимости

Исходящие: стандартная библиотека, `PyJWT[crypto]`. Входящие: `backend/app/main.py`, тесты `tests/test_auth.py`.

7. Конфигурация

`AUTH_DATABASE`, `AUTH_PRIVATE_KEY`, `AUTH_ISSUER`, `AUTH_FRONTEND_LOGIN_URL`, `AUTH_COOKIE_SECURE`, `ACCESS_TOKEN_EXPIRE_MINUTES`. Слабый/отсутствующий ключ должен ломать старт, а не деградировать в небезопасный режим.

8. Сборка и отладка

```
python -m pytest tests/test_auth.py -q          # без PostgreSQL
python -m backend.users create analyst@example.com
```

9. Тесты

`tests/test_auth.py` — создание пользователя, вход, проверка и истечение токена, отказ при неверных данных. Независимость от аналитики — сама возможность запуска без PostgreSQL.

10. Наблюдаемость

Неуспешные входы логируются без пароля и без полного токена. Метрики: число 401 по маршрутам (`/metrics`).

11. Ограничения и готовность

- Нет MFA и OTP для веб-входа (OTP есть только в Telegram-сценарии).
- Нет политики сложности пароля на уровне сервиса.
- Готовность: модуль тестируется без Postgres; утечка пароля в логи исключена тестами проверки документации и кодом логирования.